

Application of Generating Function in Algorithm

1st Kehan Wu

Communication Engineering

Sustech

ShenZhen, China

12010519@mail.sustech.edu.cn

Abstract—Generating function has demonstrated the feasibility of counting subsets of certain sets under limitations by enumerating every possible subsets by combinations of the element in the sets. However, can we broaden the application of the generating function? Can generating function not only be used in counting the number of subsets but also can boost some modern algorithm. In this paper, we first introduce a certain problem in *102 Combinatorial Problems* by Titu Andreescu and Zuming Feng [1]. Then we extend such method in solving such problem to the boosting of the DFT algorithm to the FFT algorithm by reducing the Complexity from $O(N^2)$ to $O(N * \log N)$. Then we verify the algorithm by programing in the matlab simulation. We investigate the impact of FFT algorithm in harmonic analysis field relating to the accerlation of the calculation. Intensive evaluations validate the developed theory and case studies demonstrate how to apply the theory to boost the DFT algorithm system design.

Keywords—generating function, discrete math, FFT, DFT, complex number

I. INTRODUCTION

Let's start with a simple question from *102 Combinatorial Problems* by Titu Andreescu and Zuming Feng: Find the number of subsets of $\{1, 2, 3, 4, 5, \dots, 2000\}$ the sum of whose elements is divisble by 5. With these ideas in solving such generating function problem we can further look in the inner ideas of simplify the question, and eventually develope a new algorithm in boosting *Discrete Fourier Transform*.

II. A SIMPLE DISCRETE PROBLEM

A. Generating Function in Finding Subsets

In class, we have already learnt how to use generating function in finding the amount of subsets by:

$$f(x) = (1+x)(1+x^2)(1+x^3) \cdots (1+x^{2000}) \quad (1)$$

By reshaping the function, we can write the generating function in other form as following:

$$f(x) = \sum_{i=0} a_i x^i = a_0 + a_1 x + a_2 x^2 + \dots \quad (2)$$

each a_i represent for the amount of the subsets which elements' sum is i , for each element in the set 1 stands for the element is not be picked up in the subset, while x^2 indicates the element 2 is included in the subsets. By the laws of the product of $x^i \cdot x^j = x^{i+j}$, $i+j$ indicates the elements in the subsets is already sum up, as a result a_i can be the number of the subsets which sum is i .

The problem is converted to how to find the sum of constant in the front of the x^i , which i can be written in $i = 5 \cdot k$, $k \in N$.

B. Generating Function in Finding Sum

From the analysis above, the traditional idea is to multity all the terms of $(1+x^i)$ together, and find the constant of the required form to sum them up. However, the operation of calculating the sum can be simplified by bringing the whole problem into the complex domain. When it comes to calculating the sum of the certain constant a_i , we need to shed light on an extra method of *Generating Function*. The value of x in the *Generating Function* is the key of calculating the sum of the constant we need.

Go back to the *Generating Function*:

$$f(x) = \sum_{i=0} a_i x^i = a_0 + a_1 x + a_2 x^2 + \dots$$

If $x = 1$, it means the sum of all constants will be 2^{2000}

$$f(1) = a_0 + a_1 \cdot 1 + a_2 \cdot 1^2 + \dots$$

On the other hand, we have another equation:

$$f(1) = (1+1) \cdot (1+1^2) \cdot \dots \cdot (1+1^{2000})$$

When $x = -1$,

$$f(-1) = a_0 - a_1 + a_2 - \dots$$

$$f(-1) = (1-1) \cdot (1+1) \cdot \dots = 0$$

Adding these equations together:

$$f(1) + f(-1) = 2^{2000}$$

$$f(1) + f(-1) = 2a_0 + 2a_2 + 2a_4 + \dots$$

The sum of the even constants is 2^{1999} .

Now focus on the function of $x = -1$, -1 is not only represent the value of x is -1 , but also indicates x is rotated by 180° . To be specific, x is multiplied by $e^{i\pi}$. In other words, when we need to find the constant of the even number, we can set x is solution of the unit circle with $x^2 = 1$.

So far the problem is extended to the complex domains $\mathbb{C}$, when it comes to this the problem requires constants are fixed with $5k$, the unit circle of x is divided into 5 parts, when x is rotated by 5 times, it will lead a coherent interference. On the other hand, when x isn't rotated by 5 times, it will go back to 0, owing to the incoherent interference, the sum of

the constant is 0. x is set to $x^5 = 1$. Because $x \in \mathbb{C}$, there are 5 different solutions with x .

$$x \in \left\{ e^{0i\pi}, e^{\frac{1}{5}i\pi}, e^{\frac{2}{5}i\pi}, e^{\frac{3}{5}i\pi}, e^{\frac{4}{5}i\pi} \right\}$$

Let $\zeta^i = e^{\frac{i}{5}j\pi}$. Similarly, above equations add together can

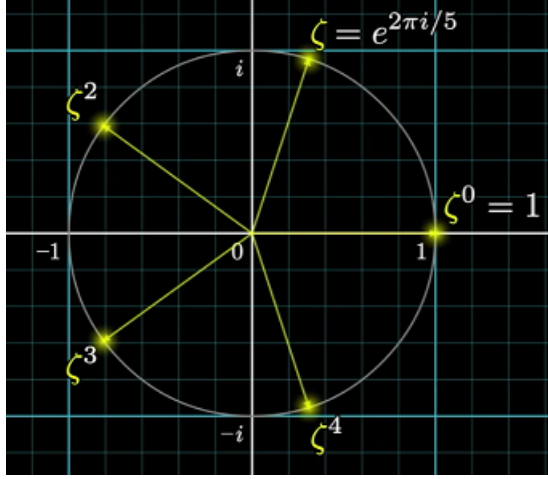


Fig. 1. x on the complex plain

find the constant we need. The character of the ζ are listed as followed:

$$\sum_i \zeta^i = 0$$

$$\zeta^{i+5} = \zeta^i$$

Next, we need to look into the sum of $f(\zeta^i)$

$$f(\zeta^0) = a_0 + a_1\zeta^0 + a_2\zeta^0 + \dots + a_5\zeta^0 + \dots$$

$$f(\zeta^1) = a_0 + a_1\zeta^1 + a_2\zeta^2 + \dots + a_5\zeta^5 + \dots$$

$$f(\zeta^2) = a_0 + a_1\zeta^2 + a_2\zeta^4 + \dots + a_5\zeta^{10} + \dots$$

$$f(\zeta^3) = a_0 + a_1\zeta^3 + a_2\zeta^6 + \dots + a_5\zeta^{15} + \dots$$

$$f(\zeta^4) = a_0 + a_1\zeta^4 + a_2\zeta^8 + \dots + a_5\zeta^{20} + \dots$$

The second column indicates that:

$$a_1(\zeta^0 + \zeta^1 + \zeta^2 + \zeta^3 + \zeta^4)$$

With the quality:

$$\zeta^0 + \zeta^1 + \zeta^2 + \zeta^3 + \zeta^4 = 0$$

While the sum of the third column:

$$\zeta^0 + \zeta^2 + \zeta^4 + \zeta^6 + \zeta^{10} = 0$$

With the character of the period, we have

$$\zeta^6 = \zeta^1, \zeta^{10} = \zeta^5$$

When it comes to the sixth column,

$$\zeta^0 + \zeta^5 + \zeta^{10} + \zeta^{15} + \zeta^{20} = 5$$

In other word, when we multiply a constant whose modular of 5 isn't zero, The sum of ζ will be zero. Assume m satisfy that $\gcd(m, 5) = 1$. After multiplied by m , which indicates m is the m th column of the equation

$$\{0, 1, 2, 3, 4\} \longrightarrow \{0, m, 2m, 3m, 4m\}$$

$$m - i = k$$

$$2m - j = k'$$

Suppose $i = j$, which indicates the modulars of m and $2m$ are the same, we have:

$$m = 5(k - kt)$$

From the equation we can find $\gcd(5, m) = 1$. Since $\gcd(5, m) \neq 1$, which is contradict with the Assumption. For each solutions, we have $i \neq j$ for each pairs of elements, so that each i can cover all elements in $\{0, 1, 2, 3, 4\}$.

The Sum of not the period of 5 will cancel eachother, as a result the Sum will only be the repetition of 5.

The value of the sum of the certain constans can be calculated from:

$$a_0 + a_5 + a_{10} + \dots = \frac{1}{5} \sum_{i=0}^4 f(\zeta^i) \quad (3)$$

Go back to the initial form:

$$f(\zeta) = ((1 + \zeta)(1 + \zeta^2)(1 + \zeta^3)(1 + \zeta^4)(1 + \zeta^5))^{400}$$

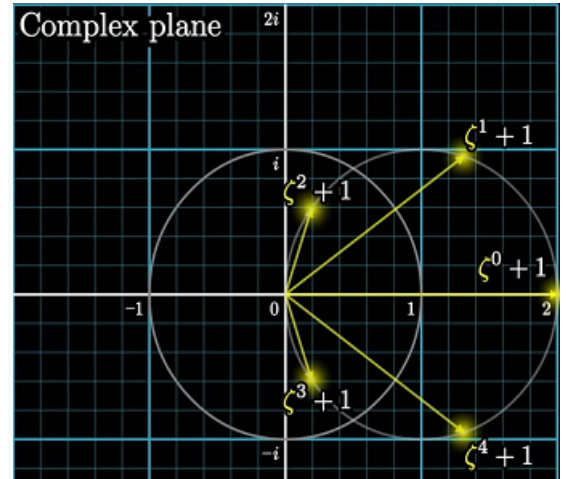


Fig. 2. $\zeta + 1$ on the complex plain

A tricky method can solve the equation. We can find that the value of the equation is 2^{400} .

$$z^5 - 1 = (z - \zeta^0) \cdot (z - \zeta^1) \cdot (z - \zeta^2) \cdot (z - \zeta^3) \cdot (z - \zeta^4)$$

$$(-1)^5 - 1 = (-1 - \zeta^0) \cdot (-1 - \zeta^1) \cdot (-1 - \zeta^2) \cdot (-1 - \zeta^3) \cdot (-1 - \zeta^4)$$

$$2 = (1 + \zeta^0) \cdot (1 + \zeta^1) \cdot (1 + \zeta^2) \cdot (1 + \zeta^3) \cdot (1 + \zeta^4)$$

$$f(\zeta) = 2^{400}$$

$$a_0 + a_5 + a_{10} + \dots = \frac{1}{5} (2^{2000} + 2^{400} + 2^{400} + 2^{400} + 2^{400})$$

C. The Sum of the Equation

So far we have: the sum of whose elements is divisble by 5 is

$$\frac{1}{5} (2^{2000} + 2^{400} + 2^{400} + 2^{400} + 2^{400})$$

III. GENERATING FUNCTION IN FAST FOURIER TRANSFORM

A. Introduction

Dating back to 1805, mathematician Carl Friendrich Gauss developed a method of harmonic analysis to determine the orbit of Juno [2]. Later during World Nuclear Forces, Scientists need a simiplified algorithm to achieve harmonic analysis which devised a novel approach to harmonic analysis same as the one by Gauss [3].

Above-mentioned question can be solved by the Fast Fourier Transform (FFT) algorithm, since they are the same form of the question.

$$f(x) = \sum_{i=0}^{n-1} a_i x^i$$

B. Discrete Fourier Transform

DFT algorithm converts a finite sequence of equally-spaced samples of a function into a same-length sequence of equally-spaced samples of the discrete-time Fourier transform (DTFT), which is a complex-valued function of frequency.

Given a series of amplitude of the signal, DFT is a tranditional algorithm to analysis the frequency component of the input signal with the equations below:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi \frac{kn}{N}}, \quad k = 0, 1, \dots, N-1 \quad (4)$$

$$= \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n} \Big|_{\omega=\frac{2\pi k}{N}} = X(e^{j\omega}) \Big|_{\omega=2\pi k/N}$$

$$\mathbf{W}_N = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & e^{-j2\pi \frac{1}{N}} & e^{-j2\pi \frac{2}{N}} & \dots & e^{-j2\pi \frac{(N-1)}{N}} \\ 1 & e^{-j2\pi \frac{2}{N}} & e^{-j2\pi \frac{4}{N}} & \dots & e^{-j2\pi \frac{2(N-1)}{N}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & e^{-j2\pi \frac{N-1}{N}} & e^{-j2\pi \frac{2(N-1)}{N}} & \dots & e^{-j2\pi \frac{(N-1)(N-1)}{N}} \end{bmatrix}$$

$$\mathbf{W}_N \cdot \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ \vdots \\ x[N-1] \end{bmatrix} = \begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ \vdots \\ X[N-1] \end{bmatrix}$$

Where $X[k]$ is the k th frequency component of the input signal, $x[n]$ is the n th amplitude of the input signal, N is the length of the input signal.

The main idea of DFT algorithm is same as the question above, the key is to find the required frequency component by coherent interference and cancel the other frequency by incoherent interference. By analysis the complexity of the DFT algorithm, the total complexity is $O(N^2)$ which is unaffordable.

C. Fast Fourier Transform

FFT algorithm rapidly computes such transformations by factorizing the DFT matrix into a product of sparse (mostly zero) factors. As a result, it manages to reduce the complexity of computing the DFT from $O(N^2)$, which arises if one simply applies the definition of DFT, to $O(N \log N)$, where N is the data size. The difference in speed can be enormous, especially for long data sets where N may be in the thousands or millions [4].

With the character of the twiddle factor W_N we have:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad k = 0, 1, \dots, N-1 \quad (5)$$

$$\text{where } W_N = e^{-j2\pi/N}$$

Such twiddle factor can be veiwed as the unit root of the circle on the complex plane, then adding other character of the twiddle factor, we can simplify the harmonic analysis:

$$\omega_n^k = \omega_{2n}^{2k}$$

$$\omega_n^{k+\frac{n}{2}} = -\omega_n^k$$

$$\omega_n^0 = \omega_n^n$$

First of all, the main point of harmonic analysis is computing this equation:

$$f(x) = \sum_{i=0}^{n-1} a_i x^i$$

then we can divide the equation into two parts, the odd part and the even part:

$$f_1(x) = a_0 + a_2 x^1 + a_4 x^2 + \dots + a_{n-2} x^{\frac{n}{2}-1}$$

$$f_2(x) = a_1 + a_3 x + a_5 x^2 + \dots + a_{n-1} x^{\frac{n}{2}-1}$$

$$f(x) = f_1(x^2) + x f_2(x^2)$$

Suppose that $n = 2^k$, $x = W_N^k$. Then we can get:

$$\begin{aligned} f(\omega_n^k) &= f_1(\omega_n^{2k}) + \omega_n^k f_2(\omega_n^{2k}) \\ &= f_1(\omega_{\frac{n}{2}}^k) + \omega_n^k f_2(\omega_{\frac{n}{2}}^k) \end{aligned}$$

Also, take $x = W_N^{k+\frac{n}{2}}$ then we have:

$$\begin{aligned} f(\omega_n^{k+\frac{n}{2}}) &= f_1(\omega_n^{2k+n}) + \omega_n^{k+\frac{n}{2}} f_2(\omega_n^{2k+n}) \\ &= f_1(\omega_n^{2k} \cdot \omega_n^n) - \omega_n^k f_2(\omega_n^{2k} \cdot \omega_n^n) \\ &= f_1(\omega_n^{2k}) - \omega_n^k f_2(\omega_n^{2k}) \\ &= f_1(\omega_{\frac{n}{2}}^k) - \omega_n^k f_2(\omega_{\frac{n}{2}}^k) \end{aligned}$$

From the results above, we can use $f_1(\omega_{\frac{n}{2}}^k)$ and $\omega_n^k f_2(\omega_{\frac{n}{2}}^k)$ to calculate the $f(\omega_n^k)$ and $f(\omega_n^{k+\frac{n}{2}})$, which means we can use the 2 results of $\frac{N}{2}$ FFT to calculate the result of N FFT. Then we can use the recursive method to calculate the result. The complexity of the FFT algorithm is $O(N \log N)$ which is much better than the DFT algorithm.

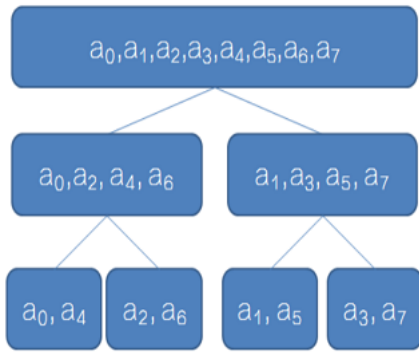


Fig. 3. Divide and Conquer with $n = 8$

$$\begin{aligned}
 X_1(k) &= X_3(k) + W_{N/2}^k X_4(k) \\
 X_1(k + \frac{N}{4}) &= X_3(k) - W_{N/2}^k X_4(k) \\
 X_2(k) &= X_5(k) + W_{N/2}^k X_6(k) \\
 X_2(k + \frac{N}{4}) &= X_5(k) - W_{N/2}^k X_6(k)
 \end{aligned}$$

Take $n = 8$ as an example, we can get the flow chart of the FFT algorithm as Figure 6.

FFT requires $2 \cdot 8 = 16$ multiplies and $3 \cdot 8 = 24$ adds. DFT requires $8^2 = 64$ multiplies and $8^2 - 8 = 56$ adds. FFT requires $N(\log_2 N - 1)$ multiplies and $N \log_2 N$ adds, while DFT requires N^2 multiplies and $N(N - 1)$ adds.

D. The order of the FFT algorithm

Next step is how to calculate the order of the FFT algorithm. We can use the divide and conquer method to calculate the order of the FFT algorithm. We can derive the order from the reverse order of the input sequence.

Actually, there two kinds of order of FFT algorithm, the first one is the Decimation by time, which means the input sequence is first separated, then all frequency values appear in order. The second one is the Extract by frequency, which means the input sequence is in the initial order, then we need to reverse the output frequency. All in all, either the order of the input or the output is the reverse order.

Here we take the Decimation by time as an example. We can derive the order from the reverse order of the input sequence. First convert the bit information into binary and then perform code bit reverse order processing.

0	1	2	3	4	5	6	7
000	001	010	011	100	101	110	111
0	4	2	6	1	5	3	7
000	100	010	110	001	101	011	111

Fig. 4. Reverse of $n = 8$

E. Butterfly Computation

The butterfly computation is the core of the FFT algorithm. It combines the results of smaller discrete Fourier transforms (DFTs) into a larger DFT, or vice versa (breaking a larger DFT up into subtransforms) [5]. The name "butterfly" comes from the shape of the data-flow diagram in the radix-2 case, as described below. The earliest occurrence in print of the term is thought to be in a 1969 MIT technical report [6] [7].

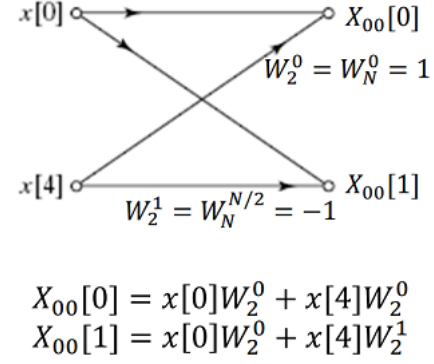


Fig. 5. Butterfly Diagram

Hence, the 8-point DFT can be obtained by the following diagram with four 2-point DFTs. Then finally we can get the result of the 8-point DFT.

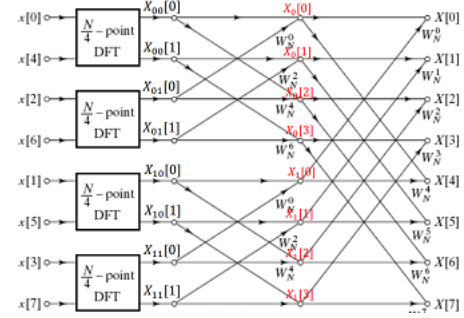


Fig. 6. Butterfly Diagram $n = 8$

IV. CONCLUSION

In this paper, we first introduce a simple problem in *102 Combinatorial Problems by Titu Andreescu and Zuming Feng*. Then we extend such method in solving such problem to the boosting of the DFT algorithm to the FFT algorithm by reducing the Complexity from $O(N^2)$ to $O(N * \log N)$. Then we verify the the algorithm by programing in the matlab simulation. We investigate the impact of FFT algorithm in the harmonic analysis relating to the accerlation of the calculation. Intensive evaluations validate the developed theory and case studies demonstrate how to apply the theory to boost the DFT algorithm system design.

The core between these question is the idea of bringing the problem into the Complex Domain, it's hard to link the

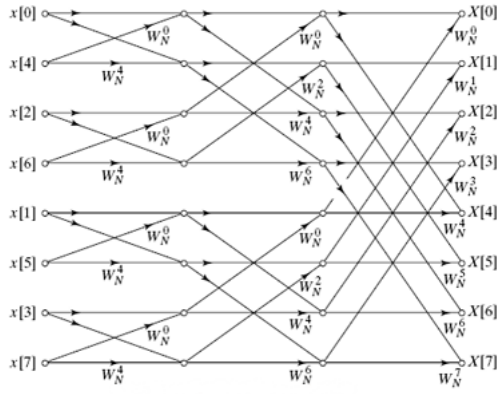


Fig. 7. Butterfly Diagram $n = 8$ with time

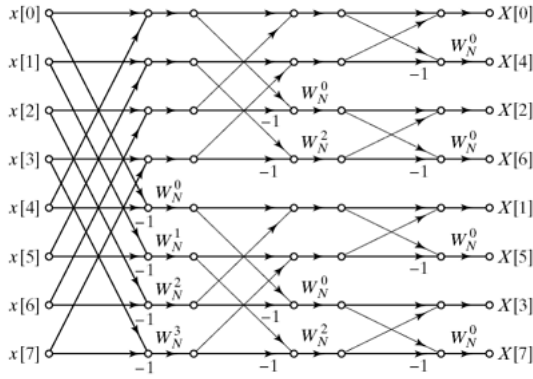


Fig. 8. Butterfly Diagram $n = 8$ with frequency

problem with the Complex Domain in the first simple question. Finding the subsets of a certain set under such constrain is a very common problem in the Combinatorial Problems, but it's hard to find the relationship between the Complex Domain and the Combinatorial Problems. Since the Complex Domain is a very powerful tool in solving the Discrete Problems, we can extend the problem to the Complex Domain to

Generating Function is a convenient bridge crossing the Discrete Domain and the Complex Domain. Selecting a proper x on the Complex Plane, using the rotation of the Complex Number to represent the Discrete Number, then the coherent and the incoherent interference can cancel and strengthen the component required in the problem. The character of the unit circle is the key to simplify complexity of the operation. To be specific, this kind of method impact the polynomial calculation deeply.

V. ACKNOWLEDGMENT

Throughout the writing of this dissertation I have received a great deal of support and assistance.

I would first like to thank my supervisor, Ming Tang, whose expertise was invaluable in formulating the research questions and methodology. Your insightful feedback pushed me to sharpen my thinking and brought my work to a higher level.

I would particularly like to acknowledge my teammate/group mate/team members, LASSO, for their wonderful collaboration and patient support

I would also like to thank my tutors, Rui Wang, for their valuable guidance throughout my studies. You provided me with the tools that I needed to choose the right direction and successfully complete my dissertation.

In addition, I would like to thank my parents for their wise counsel and sympathetic ear. You are always there for me.

Finally, I could not have completed this dissertation without the support of my friends, Renqi Chen, who provided stimulating discussions as well as happy distractions to rest my mind outside of my research.

REFERENCES

- [1] Andreescu, Titu., and Zuming. Feng. 102 Combinatorial Problems From the Training of the USA IMO Team. 1st ed. 2003. Boston, MA: Birkhäuser Boston, 2003. Web.
- [2] Heideman, Michael T., Don H. Johnson, and C. Sidney Burrus. "Gauss and the History of the Fast Fourier Transform." *Archive for history of exact sciences* 34.3 (1985): 265–277. Web.
- [3] J. Cooley, R. Garwin, C. Rader, B. Bogert and T. Stockham, "The 1968 Arden house workshop on fast Fourier transform processing," in *IEEE Transactions on Audio and Electroacoustics*, vol. 17, no. 2, pp. 66-76, June 1969, doi: 10.1109/TAU.1969.1162047.
- [4] Heideman, Michael T.; Johnson, Don H.; Burrus, Charles Sidney (1984). "Gauss and the history of the fast Fourier transform". *IEEE ASSP Magazine*. 1 (4): 14–21. CiteSeerX 10.1.1.309.181. doi:10.1109/MASSP.1984.1162257. S2CID 10032502. Archived from the original on 2013-03-19.
- [5] Alan V. Oppenheim, Ronald W. Schaffer, and John R. Buck, *Discrete-Time Signal Processing*, 2nd edition (Upper Saddle River, NJ: Prentice Hall, 1989)
- [6] C. J. Weinstein (1969-11-21). *Quantization Effects in Digital Filters* (Report). MIT Lincoln Laboratory. p. 42. Archived from the original on February 11, 2015. Retrieved 2015-02-10. This computation, referred to as a 'butterfly'
- [7] Cipra, Barry A. (2012-06-04). "FFT and Butterfly Diagram". *mathoverflow.net*. Retrieved 2015-02-10.

VI. APPENDIX

Codes for the FFT algorithm is packed in the appendix.